IT4931 / IT5384 / IT4043E · SOICT · HUST

Presto / Trino

Lecture 7 — SQL on lakes & NoSQL stores

Architecture · connectors · execution · when to use

Learning outcomes

1. **Explain** Presto/Trino as interactive **SQL-on-anything** (not a database)
2. **Map** Facebook-style use cases: interactive BI · ETL · A/B · low-latency apps
3. **Describe** coordinator · workers · Connector API (metadata / location / source / sink)
4. **Trace** SQL → logical plan → stages → tasks → splits → pipelined shuffles
5. **Explain** layouts, pushdown, lazy/columnar reads — and **fail-fast** (not MR-style FT)
6. **Note** 2026 naming: **Trino** continues the PrestoSQL lineage; Athena built on Presto

Roadmap

1. **Motivation & use cases** — why Hive was not enough
2. **Architecture** — coordinator / workers / connectors
3. **Execution** — plan, stages, tasks, splits, shuffles
4. **Optimizations** — layouts, pushdown, columnar I/O
5. **Benchmarks & choose** — when to use

Motivation

History (Facebook → open source)

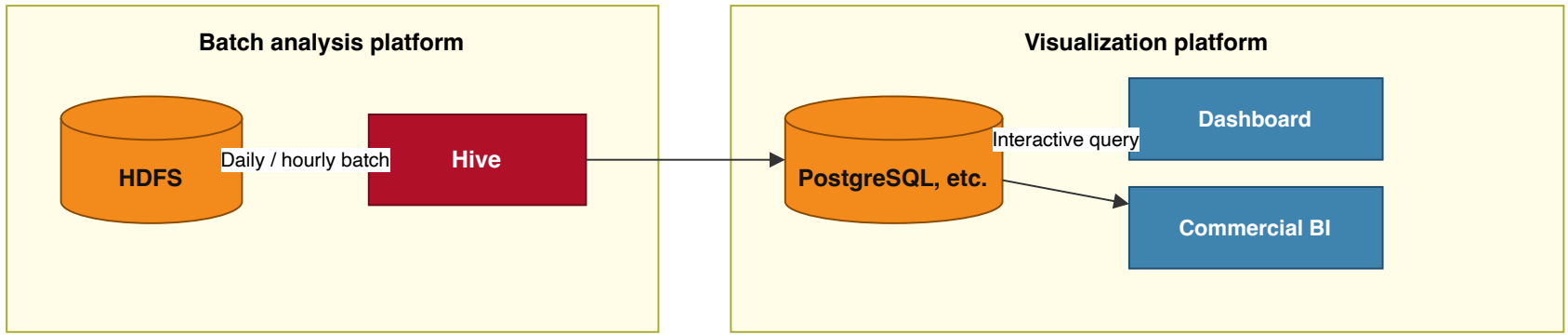
Year	Milestone
2012	Project starts at Facebook — interactive SQL on warehouse-scale data
2013	Open sourced; production at FB
2019+	Large community; lineage splits → **PrestoDB** vs **Trino** (PrestoSQL)

*Industry: Netflix, Uber, LinkedIn, … · **Amazon Athena** is built on Presto.*
*In this course we say **Presto/Trino** for the architecture family.*

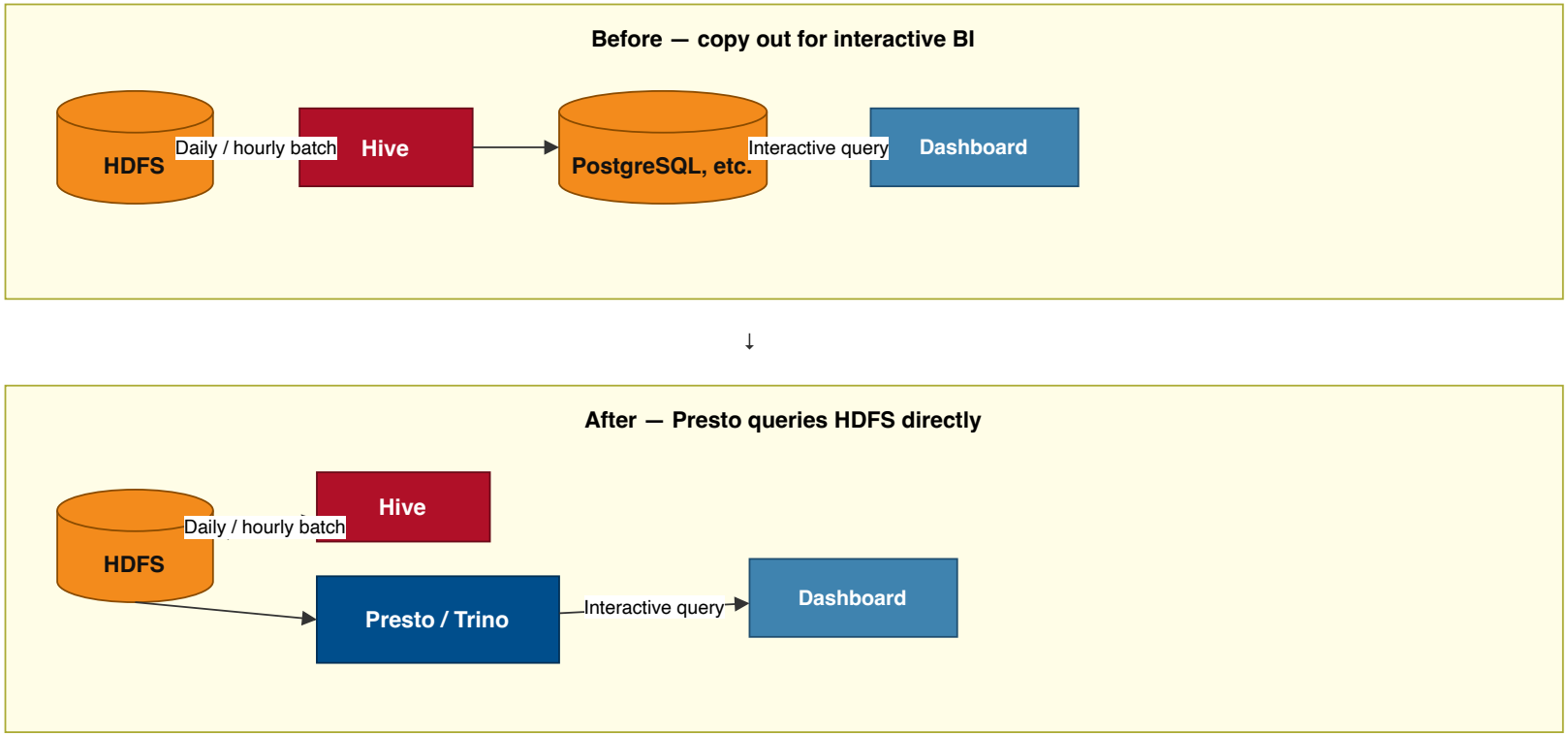
Pain: Hive too slow for BI

Dashboards need **seconds**, not MapReduce minutes — and not every source lives in HDFS.

- Hive / MR too slow (or ODBC unstable) for interactive BI
- Copying daily results into Postgres/Redshift → **costly**, less scalable
- Some data never in HDFS — still want one SQL surface



Motivation storyboard



What Presto can do

- **ANSI SQL** interactive queries (ms → minutes)
- **JDBC/ODBC** + HTTP for BI tools / dashboards
- Query **across** Hive, HBase, Cassandra, RDBMS, Kafka, … via **connectors**
- Plugin model — write your own connector in Java
- Still keep **MR/Hive/Spark** for heavy ETL when needed
- One platform for batch-ish SQL **and** interactive visualization

One engine · many use cases

Use case	What users need
Interactive analytics	Seconds–minutes; 50–100 concurrent explorers; early cancel / `LIMIT` OK
Batch ETL	Throughput & memory for TB-scale joins/aggs; latency less critical
A/B testing	Complete joins + interactive slice/dice (~5–30 s) on experiment data
App / advertiser analytics	Very low latency (~50 ms–5 s), high concurrency, selective filters

Same SQL engine — different cluster sizing, connectors, and scheduling knobs.

Deployment at Facebook (scale example)

PPTX snapshot — **Facebook, 2013** production numbers (order-of-magnitude history, not a 2026 bake-off).

Fact	Scale
Regions	Multiple geographies
Cluster	**~1,000** nodes
Users	**1,000+** employees
Queries	**30,000+** / day
Throughput	**~1 PB** / day

Who else runs Presto-family engines

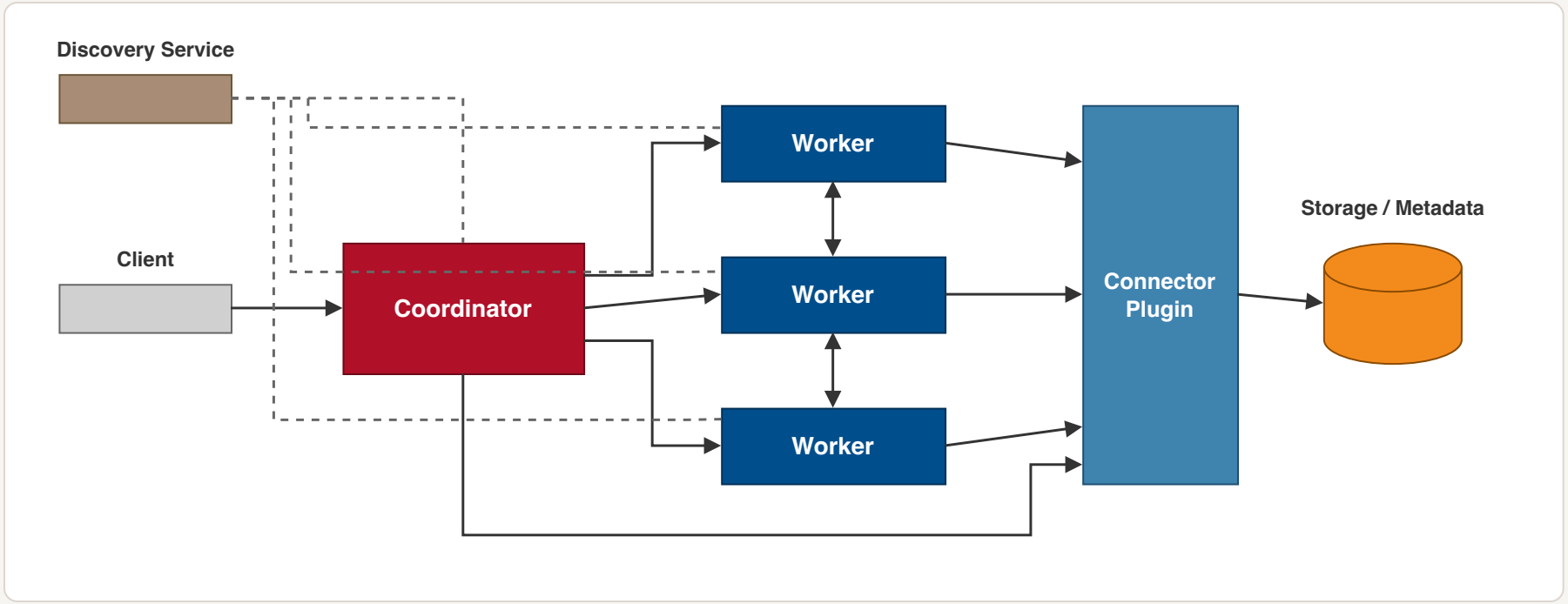
PPTX slide was a **logo wall** — teach the pattern, not the brands.

Who	Why it matters
Meta / PrestoDB	Origin; interactive SQL at PB/day scale
Trino (PrestoSQL fork)	Active open-source lineage many vendors ship
Netflix · Airbnb · Uber · LinkedIn	Early lake / warehouse-adjacent interactive SQL
AWS Athena · Starburst · …	Managed Presto/Trino over object storage

Adoption = "SQL engine over existing stores" — same idea as FB 2013, different packaging in 2026.

Architecture

Presto architecture



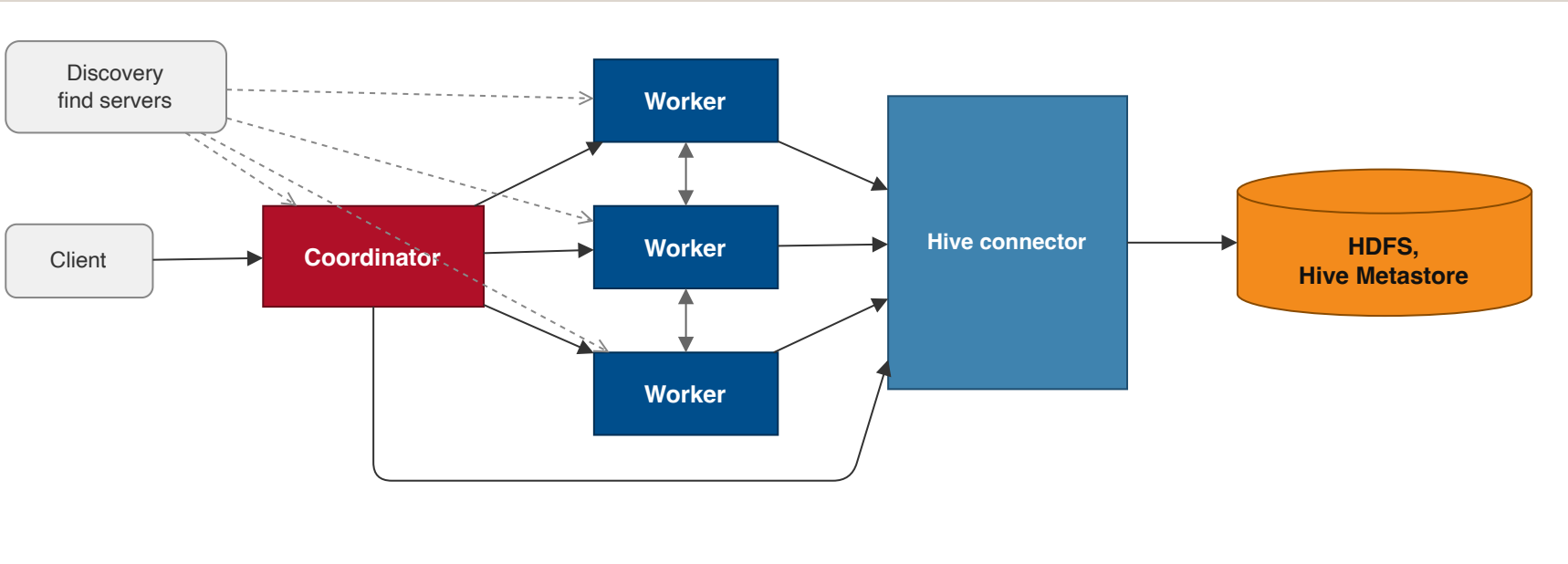
Three server roles

Role	Job
Coordinator	Parse · plan · schedule · return results
Workers	Execute tasks; in-memory shuffles
Discovery	Cluster membership (“find servers”)

Presto is NOT a database — SQL engine over **existing** stores via connectors.
Client protocol: **HTTP + JSON**. Bindings: JDBC, Python, R, Node, …

Connectors

- Java **plugins** — access storage + metadata
- Schema → **coordinator**; rows/pages → **workers**
- Built-ins: Hive, Cassandra, JDBC/MySQL, … (+ custom)

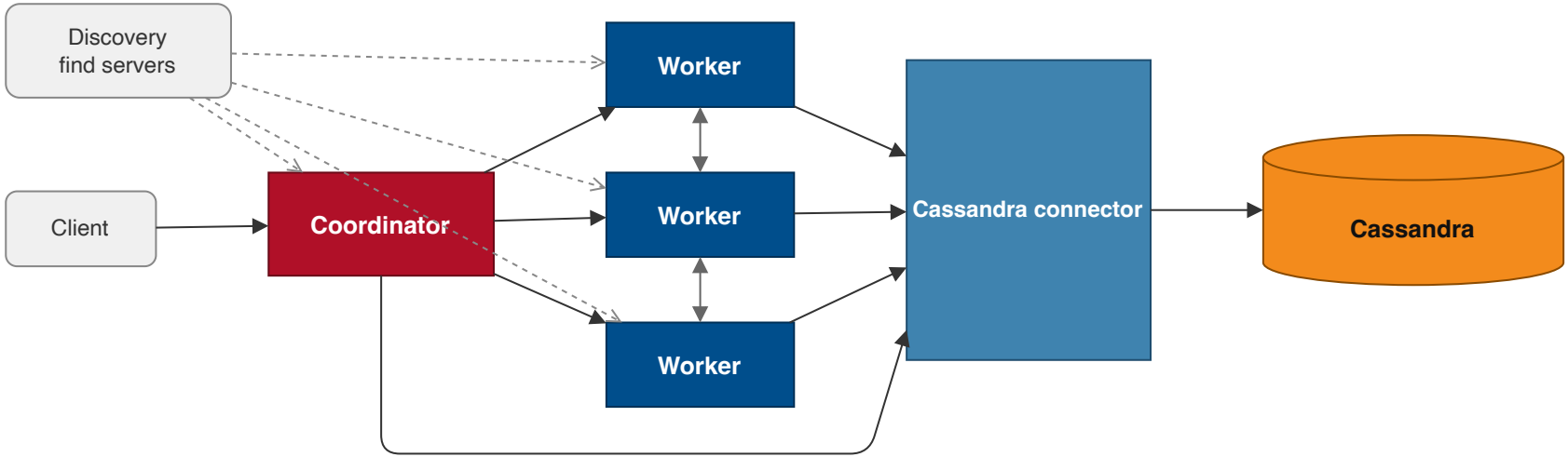


Connector API (4 surfaces)

API	Role
Metadata	Schemas, tables, columns, stats
Data location	Layouts, splits, placement hints
Data source	Read pages (columnar blocks) for a split
Data sink	Write results (ETL)

Same pattern for Hive, Cassandra, JDBC, Iceberg, …

Another connector example



Catalog · schema · table

```sql
SELECT *
FROM hive.sales.orders o
JOIN mysql.crm.customers c
  ON o.customer_id = c.id
WHERE o.order_date ⩾ DATE '2026-01-01';
```

- `catalog.schema.table` naming
- Join runs on **Presto workers** (unless connector can push work down)
- 2026: prefer **Iceberg/Delta/Hudi** catalogs over raw Hive where possible

Query execution

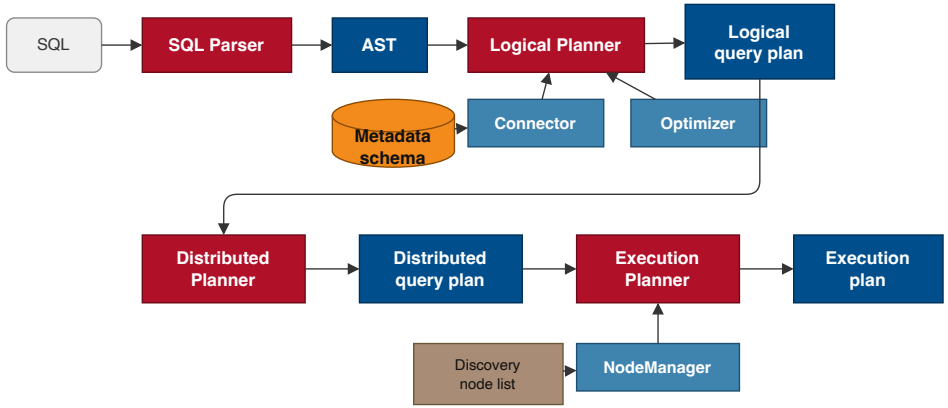
Query lifecycle (coordinator)

1. Client → **HTTP** SQL statement
2. **Parse** (ANTLR) → analyze types / functions
3. **Logical plan** — tree of operators (no placement yet)
4. **Optimize** — pushdown, layouts, join order/strategy, insert **stages** + shuffles
5. **Schedule** tasks on workers; **enumerate splits** lazily for leaf stages
6. Stream results as pages become ready (can finish early with `LIMIT`)

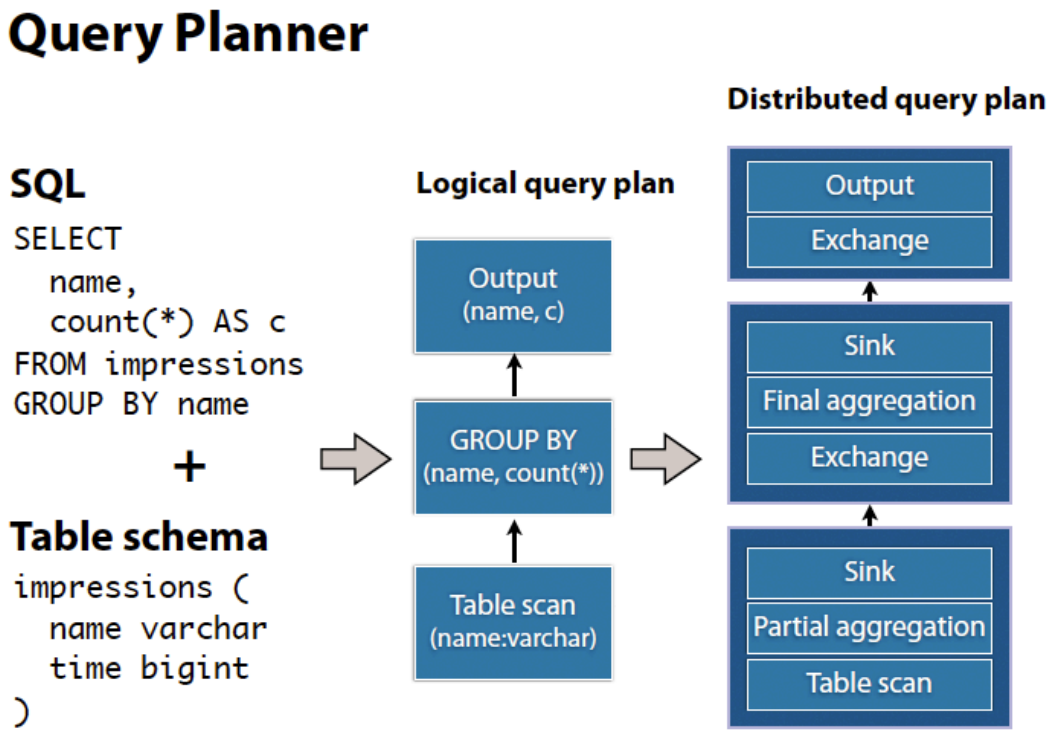
Execution model — not MapReduce

DAG / MPP-style plan — closer to Tez or classic MPP than batch MR.

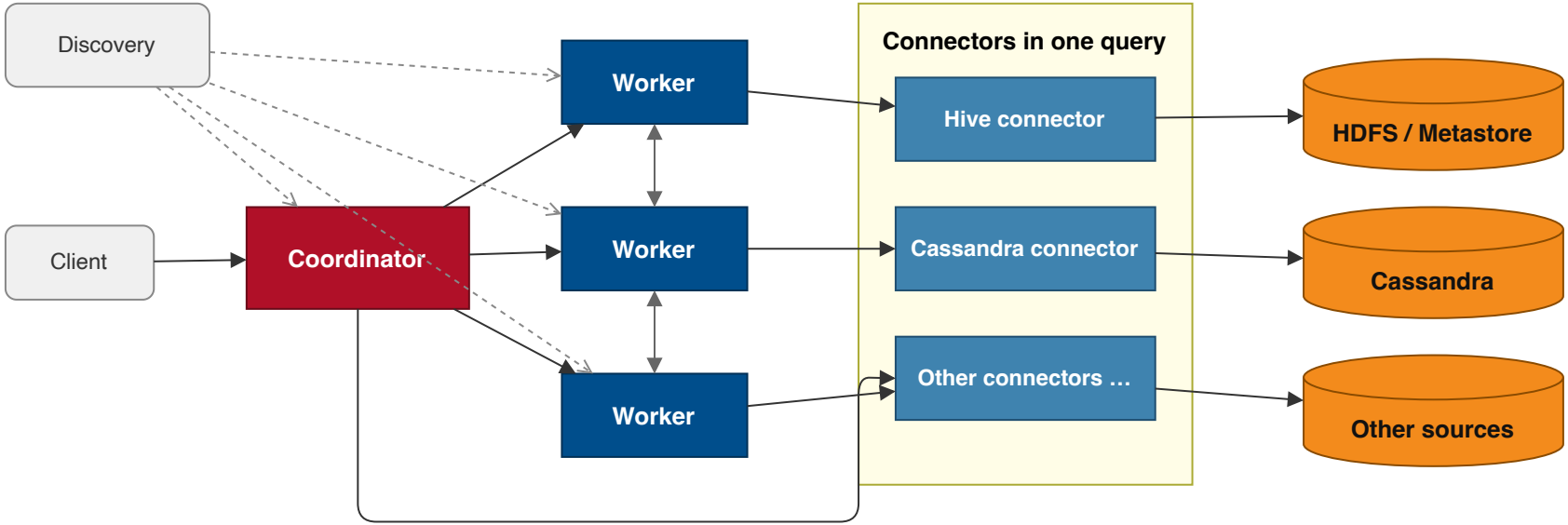
Where	Components
Coordinator	SQL parser · query planner · execution planner
Workers	Task execution · scheduler



Query planner

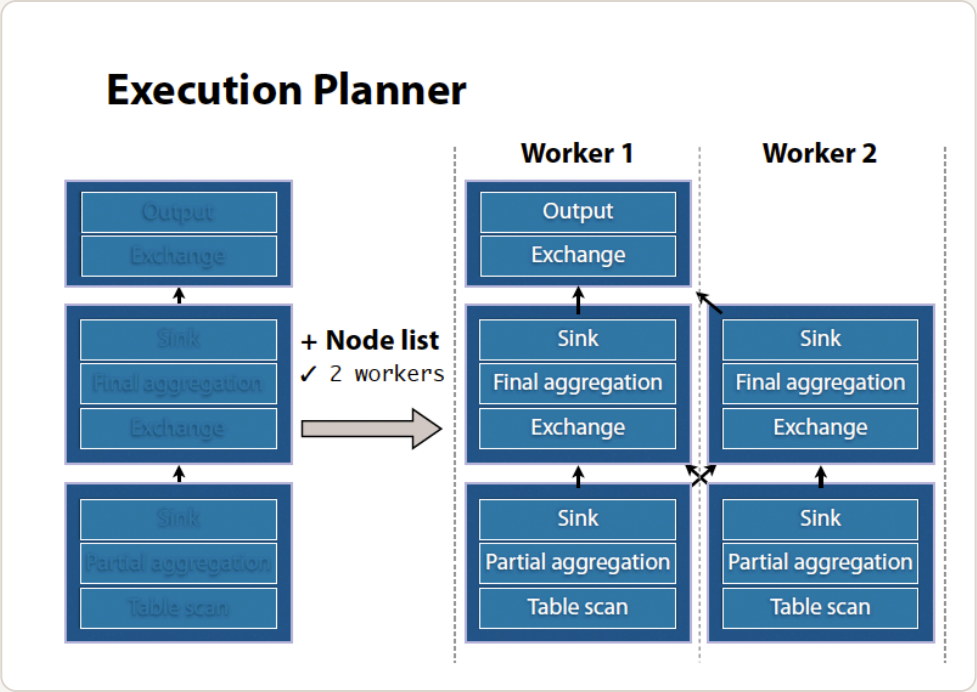


Multiple connectors in a query

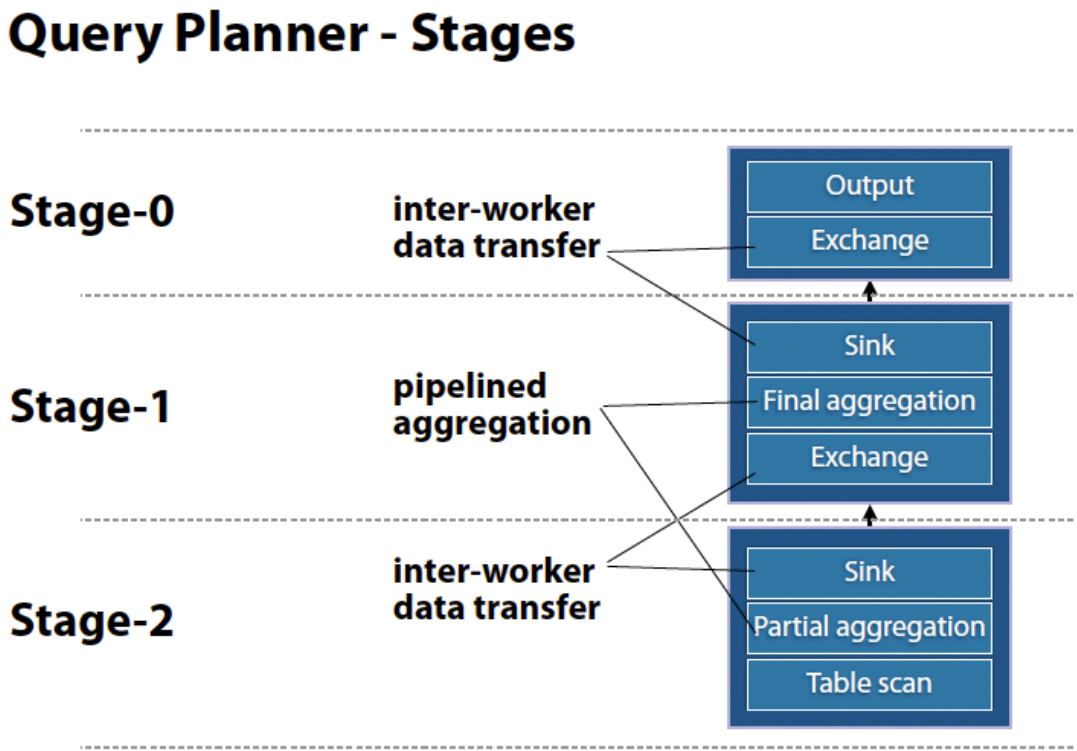


Stages · tasks · pipelines

- **Stage** — parallel fragment across workers
- Same computation, different input slices
- Shuffles = **buffered in-memory** transfers (latency + CPU + buffer)



Stages (pipelined view)

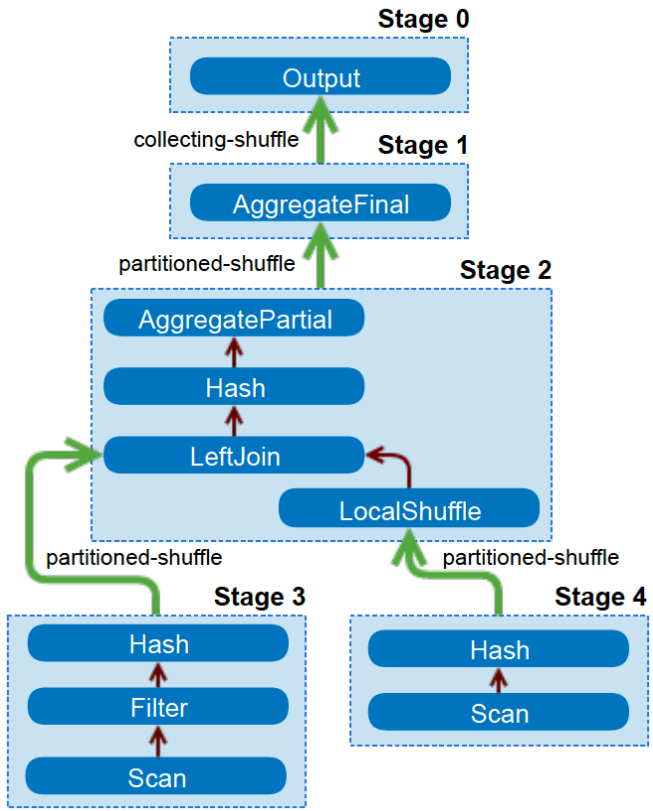


Example query → plans

```
SELECT orders.orderkey, SUM(tax)
FROM orders
LEFT JOIN lineitem
  ON orders.orderkey = lineitem.orderkey
WHERE discount = 0
GROUP BY orders.orderkey
```

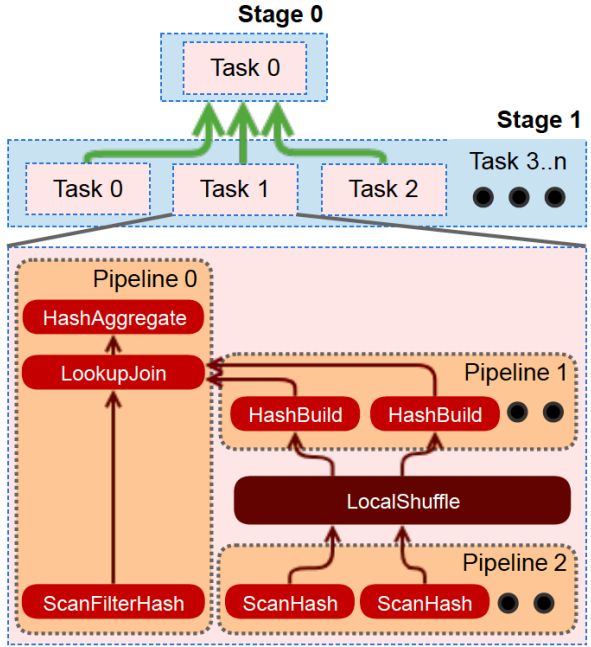
Next slide: **logical** tree → **distributed** stages + exchanges (shuffles).

Logical vs distributed plan



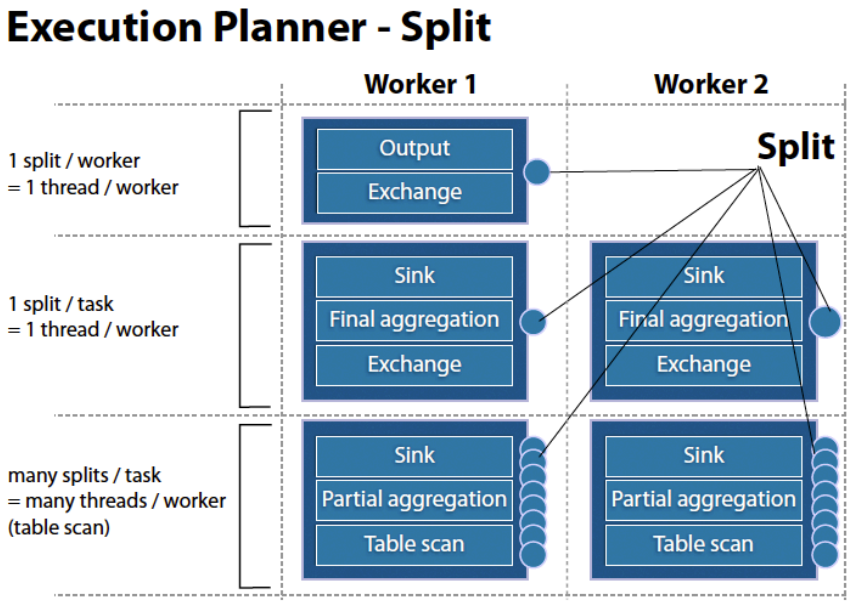
Tasks across workers

Coordinator ships stages as executable **tasks**: **1 task / worker / stage**.
A task may have multiple **pipelines**; a pipeline = chain of **operators**.



Splits

- Opaque handle to a **chunk** of data (e.g. HDFS path + offsets)
- Assigned **lazily** — enumerate as query runs
- Shortest-queue worker; can start results early (`LIMIT`)
- Many splits / task ⇒ many threads (esp. table scan)



Scheduling policies

Policy	Idea	Fit
All-at-once	Schedule all stages concurrently	Interactive / A/B / app analytics
Phased	Stage-by-stage when ready	Batch analytics · memory

Leaf stages: honor connector + network / co-location.
Intermediate stages: place on any worker; still pick task count.

Pipelined execution properties

- All tasks run in parallel — **no MR barrier** between stages
- Memory-to-memory transfer — **no disk** between stages
- If one task fails → **query fails** (fail-fast; worker may stay up)
- Aggregates must fit **memory** (query dies if not — classic Presto model)
- Memory of concurrent queries is **managed** cluster-wide

Optimizations

Data layouts

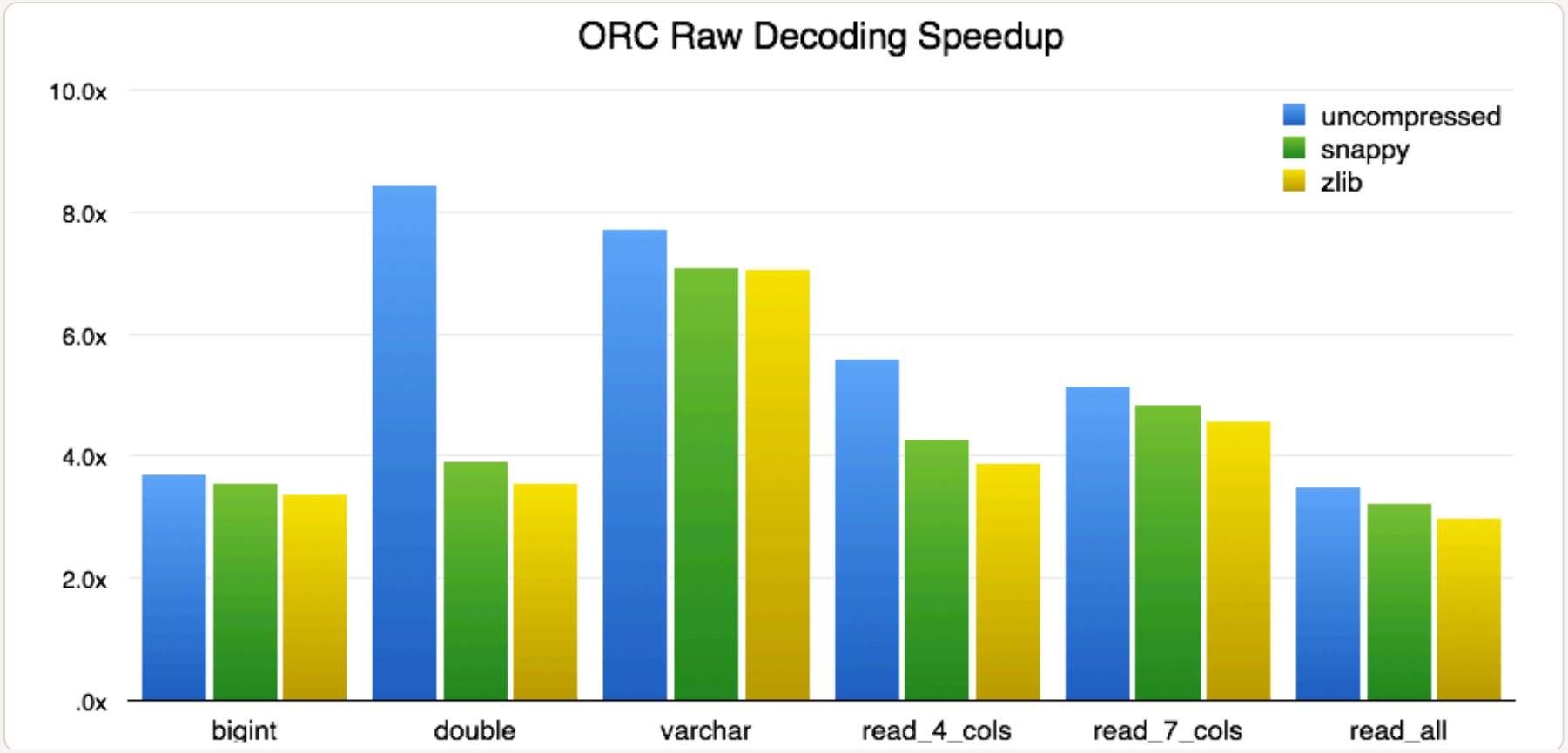
Optimizer uses physical properties: **partitioning, sorting, grouping, indexes**.

- Tables may expose **multiple layouts** (different properties)
- Layouts can be **column subsets** / partial data
- Optimizer picks best layout for the query
- Tuning often = add better lake layouts (format / partition / clustering)

Predicate pushdown

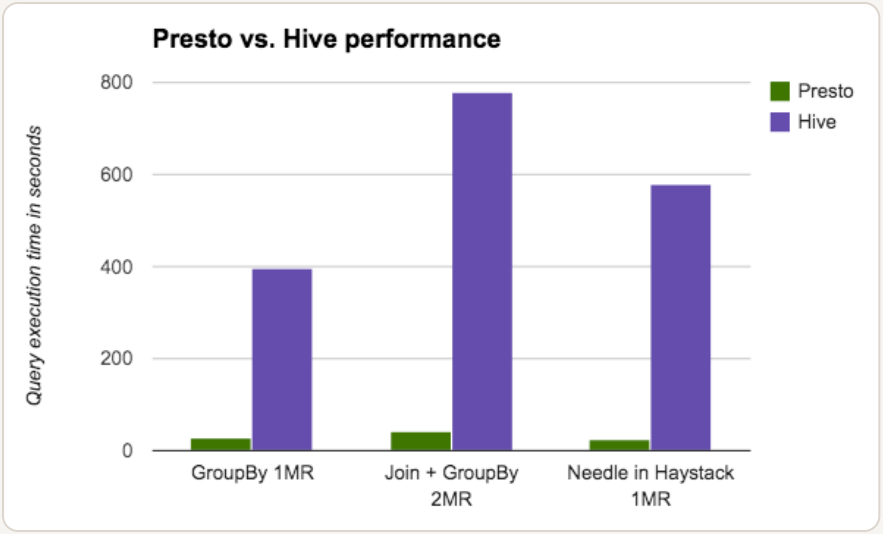
- Push equality/range predicates **into connectors**
- Engine gives a two-part constraint:
  - **Domain** — ranges + nullability
  - Remaining **“black box”** predicate
- Huge win on columnar formats (ORC/Parquet) + partition pruning

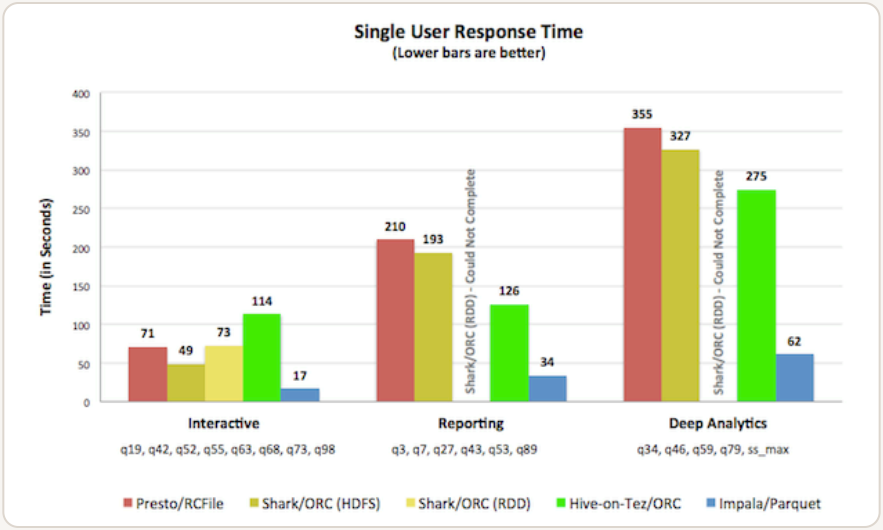
ORC reader improvements



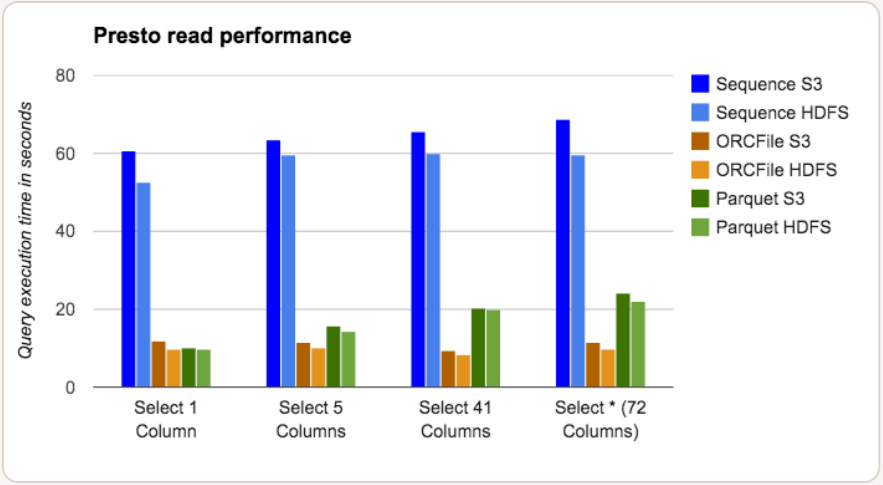
Industry benchmarks

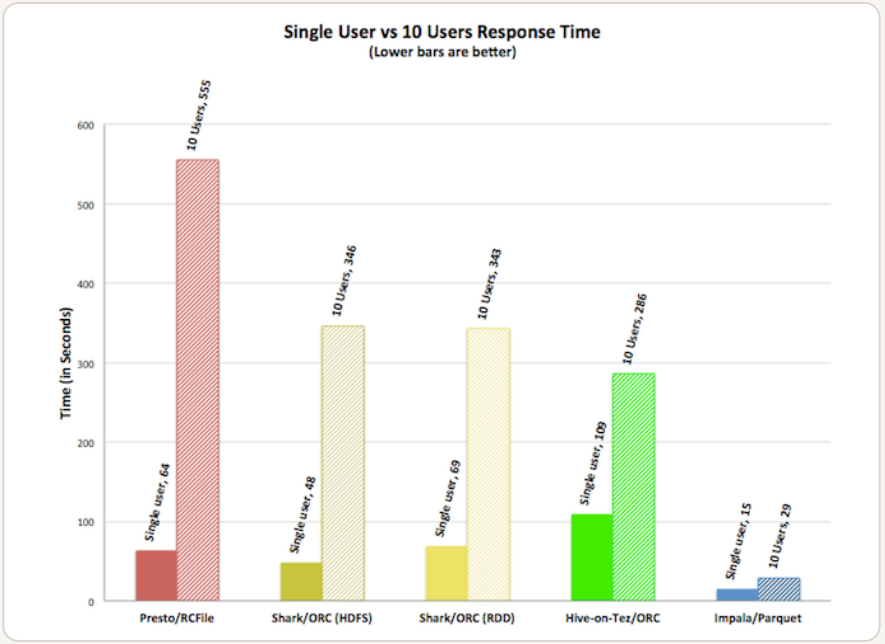
Industry benchmark examples





More benchmark slides

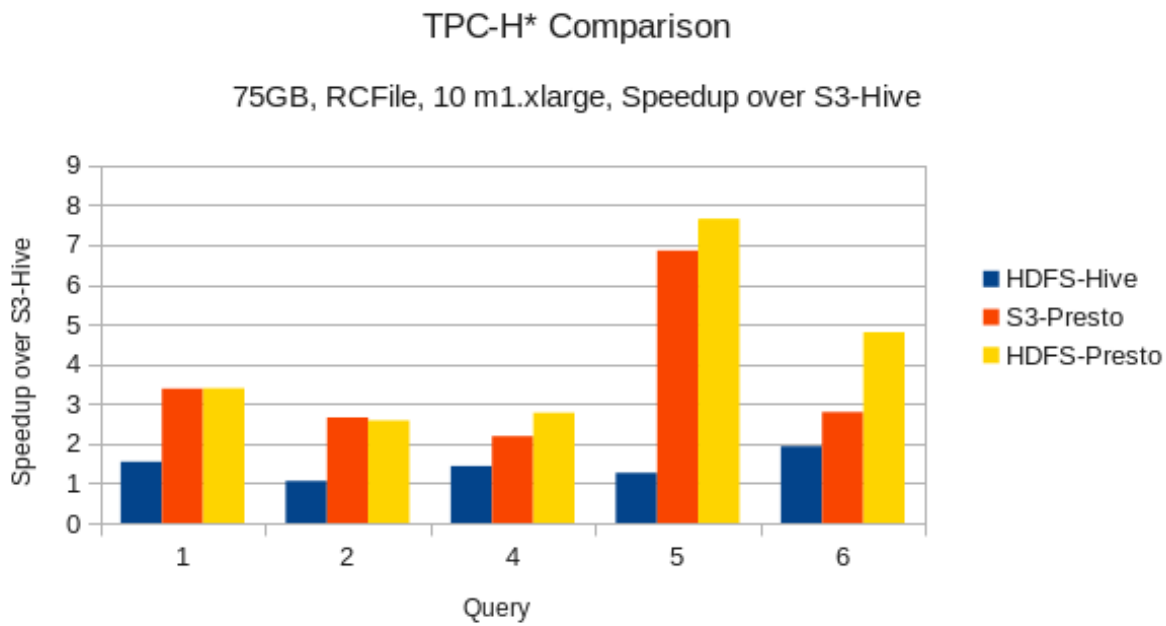




Benchmark figures are illustrative; focus on architecture takeaways.

Qubole TPC-H snapshot

Historical vendor chart: Presto vs Hive speedup on ~75 GB RCFile — treat as **order-of-magnitude**, not a current bake-off.



When to use Presto/Trino

Prefer Presto/Trino	Prefer something else
Interactive SQL / BI on lake	Heavy ETL / ML training → **Spark**
Federate many sources	Strong OLTP → **RDBMS / DynamoDB**
Decoupled storage+compute SQL	Fully managed warehouse SLA → **BQ/Snowflake/…**

Course links

- **L02–L03** — HDFS / lake storage underneath
- **L04 / L14** — lakehouse tables · serving Gold via Trino/BI
- **L05–L06** — NoSQL / DynamoDB as connector sources
- **L08** — Kafka (streaming path into the lake)
- **L10** — Spark SQL for pipelines vs interactive federation

Takeaways

1. Presto/Trino = **distributed SQL engine**, not a store
2. **Coordinator + workers + connectors**
3. **Pipelined MPP** stages/tasks/splits — fail-fast, memory-aware
4. Speed comes from **pushdown + layouts + columnar formats**
5. Use for **interactive federation**; Spark/warehouses for other jobs

Next

L08 — Apache Kafka